python crawler Tutorial

1.robots.txt

`robots.txt` is a text file placed in a website's root directory (e.g., `https://www.imdb.com/robots.txt`). It defines **rules for crawlers** (which pages to crawl/avoid) — a "gentlemen's agreement" between websites and crawlers (not legally binding, but violating it may lead to IP bans or legal risks).

Step 1: How to View a Website's `robots.txt`

To check IMDB's crawler rules, simply visit:

```
https://www.imdb.com/robots.txt
```

Step 2: Key Components of `robots.txt`

A typical `robots.txt` file uses two core directives:

- `User-agent`: Specifies which crawler the rule applies to (use `*` for all crawlers).
- `Disallow`: Lists pages/paths **prohibited** for crawling.
- `Allow`: Lists pages/paths **allowed** for crawling (overrides `Disallow` if conflicting).

Step 3: Example Analysis (IMDB's `robots.txt`)

IMDB's `robots.txt` includes rules like:

```
# robots.txt for https://www.imdb.com properties
# See https://m.imdb.com/robots.txt for the mDot equivalent

User-agent: *
Disallow: /OnThisDay
Disallow: /*/OnThisDay
Disallow: /ads/
Disallow: /*/ads/
Disallow: /ap/
Disallow: /*/ap/
Disallow: /mymovies/
Disallow: /*/mymovies/
Disallow: /register
Disallow: /*/register
Disallow: /registration/
Disallow: /*/registration/
......

User-agent: Baiduspider
Disallow: /list/*
Disallow: /*/list/*
Disallow: /user/*
Disallow: /*/user/*
```

```
User-agent: bingbot
Disallow: /showtimes/location/*
Disallow: /*/showtimes/location/*

User-agent: anthropic-ai
Disallow: /
User-agent: Claude-Web
Disallow: /
User-agent: CCbot
Disallow: /
User-agent: FacebookBot
Disallow: /
User-agent: Google-Extended
Disallow: /
User-agent: GPTBot
Disallow: /
User-agent: PiplBot
Disallow: /
```

interpretation:

- All crawlers (`User-agent: *`) **cannot** crawl `/ads/`, `/ap/`, or `/mymovies/`.
- BaiduSpider is **prohibited** from crawling:
  - Pages under `/list/` (e.g., `https://example.com/list/movies`)
  - Nested `/list/` paths (e.g., `https://example.com/category/list/top10`)
  - User-related pages (e.g., `https://example.com/user/profile`, `https://example.com/group/user/posts`)
- Crawlers like Anthropic's AI, GPTBot, Claude-web, CCBot, FacebookBot, Google-Extended, and PipIBot are **completely blocked**
- Crawlers **can** crawl `/title/` (movie detail pages, e.g., `https://www.imdb.com/title/tt15398776/`).

Step 4: Critical Notes for Compliance

1. **Always check** `robots.txt` **first** before crawling a website.
2. **Respect** `Disallow` **rules** — avoid crawling prohibited pages (e.g., IMDB's search/list pages).
3. **No legal force, but high risk**: Even if `robots.txt` isn't legally binding, violating it may result in IP bans or legal action (especially for commercial use).

中华人民共和国刑法（285. 286. 287）

信息发布时间：2007-08-07　　字体：【 大 中 小】

第二百八十五条　违反国家规定，侵入国家事务、国防建设、尖端科学技术领域的计算机信息系统的，处三年以下有期徒刑或者拘役。

第二百八十六条　违反国家规定，对计算机信息系统功能进行删除、修改、增加、干扰，造成计算机信息系统不能正常运行，后果严重的，处五年以下有期徒刑或者拘役；后果特别严重的，处五年以上有期徒刑。

违反国家规定，对计算机信息系统中存储、处理或者传输的数据和应用程序进行删除、修改、增加的操作，后果严重的，依照前款的规定处罚。

故意制作、传播计算机病毒等破坏性程序，影响计算机系统正常运行，后果严重的，依照第一款的规定处罚。

第二百八十七条　利用计算机实施金融诈骗、盗窃、贪污、挪用公款、窃取国家秘密或者其他犯罪的，依照本法有关规定定罪处罚。

我国《刑法》第285-287条明确规定：

- 非法获取计算机信息系统数据罪
- 非法控制计算机信息系统罪
- 提供侵入、非法控制计算机信息系统程序、工具罪

《网络安全法》第27条：

> 任何个人和组织不得从事非法侵入他人网络、干扰他人网络正常功能、窃取网络数据等危害网络安全的活动

真实案例警示录

案例 1：某招聘网站爬虫案

某公司使用分布式爬虫每秒发起 500 次请求，导致目标网站瘫痪 3 小时。最终判决结果：

- 赔偿经济损失 80 万元
- 技术负责人被判有期徒刑 1 年（缓刑）

案例 2：电商价格监控案

某公司在爬取竞品价格数据时，绕过了网站的签名验证机制。法院认定其构成“破坏计算机信息系统罪”，并处以 200 万元罚款

中文部分引自：CSDN 博主「kernelguru」原创文章，及济宁市人民政府官网。

原文链接（CSDN）：https://blog.csdn.net/kernelguru/article/details/148049096

（济宁市人民政府）：中华人民共和国刑法（285.286.287）

2.Developer Mode

Browser Developer Mode (also called Developer Tools) is a built-in set of debugging tools in modern browsers (Chrome, Firefox, Edge, etc.). It is the **core tool** for analyzing web page structure and locating target data when writing crawlers — it helps you quickly find the HTML tags/attributes where movie data (e.g., title, rating) is stored on IMDB pages.

Step 1: Open Developer Tools

There are 3 common ways to launch Developer Tools (take Chrome as an example):

1. **Keyboard shortcut** (most efficient): Press `F12` (Windows/Linux) or `Cmd + Opt + I` (Mac).
2. **Right-click menu**: Right-click on any area of the target web page → select `Inspect` (or "检查" in Chinese).
3. **Browser menu**: Click the three-dot icon in the top-right corner → `More tools` → `Developer tools`.

Step 2: Core Panels for Crawler Development

The Developer Tools interface has multiple panels; focus on these 3 for crawler work:

Panel	Key Functions for Crawling
Elements	View/Edit HTML structure of the page; locate tags for target data (e.g., movie title/rating).
Network	Monitor all HTTP requests/responses (e.g., check if IMDB pages use API to load data).
Source	Inspect/debug raw source code (HTML/JS/CSS) of the page; set breakpoints to analyze dynamic data loading logic; locate embedded JSON data (e.g., **NEXT_DATA** in IMDB pages) used for rendering content.

Step 3: Practical Use (Locate IMDB Movie Data)

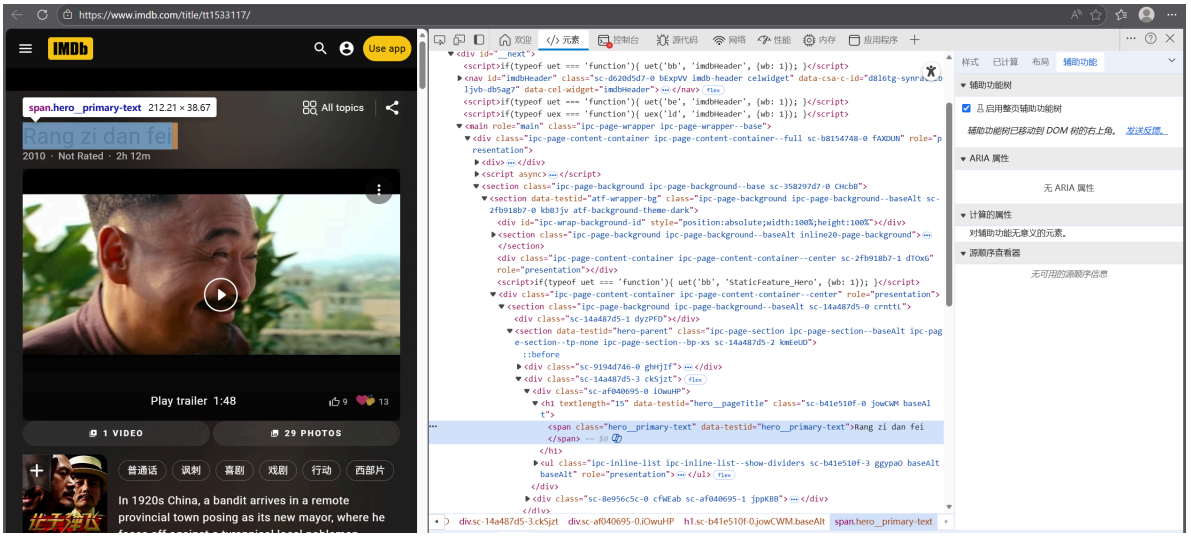
Let's use IMDB's movie detail page (e.g., `https://www.imdb.com/title/tt1533117/`) to demonstrate how to find data tags — this page features *Let the Bullets Fly* (my favorite movie, directed by Jiang Wen).

3.Locate Core Data Tags (Practical Demonstration)

We'll target key movie fields (title, director, rating, cast, release date, etc.) and find their corresponding HTML tags using the "Inspect Element" feature:

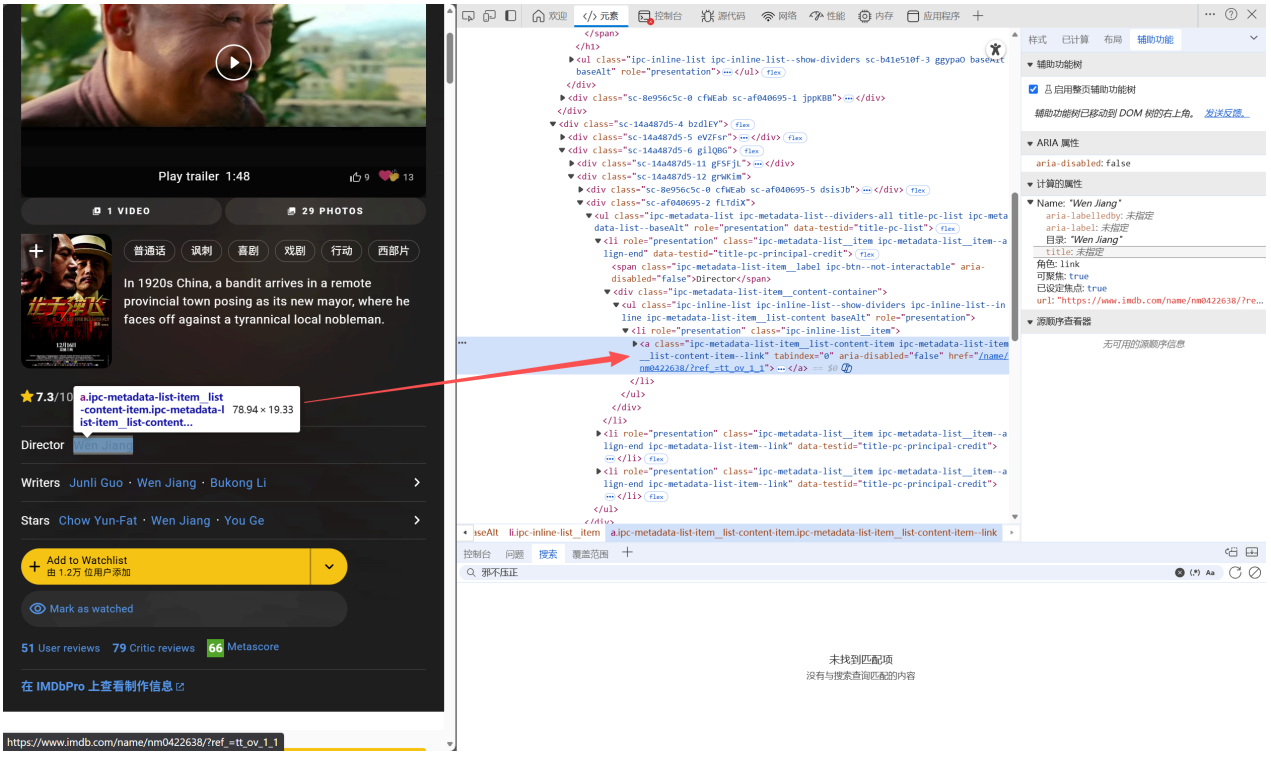
3.1 Locate Movie Title

- **Goal**: Find the tag for the title (*Rang zi dan fei*) and original title (*Rang zi dan fei*).
- **Operation**:
    1. Hover over the large title at the top of the page (e.g., "Rang zi dan fei").
    2. Right-click the title → Select **Inspect**.



3.2 Locate Director (Jiang Wen)

- **Goal**: Find the tag for the director "Wen Jiang" (Jiang Wen).
- **Operation**:
    1. Scroll down the page to the "Director" section.
    2. Hover over "Wen Jiang" → Right-click → Select **Inspect**.



3.3 Locate IMDB Rating (7.3/10)

> do it yourself!

3.4 Locate Key Cast (Chow Yun-Fat, You Ge, etc.)

> do it yourself

3.5 Locate Country, Year_released, run_time etc

> do it yourself

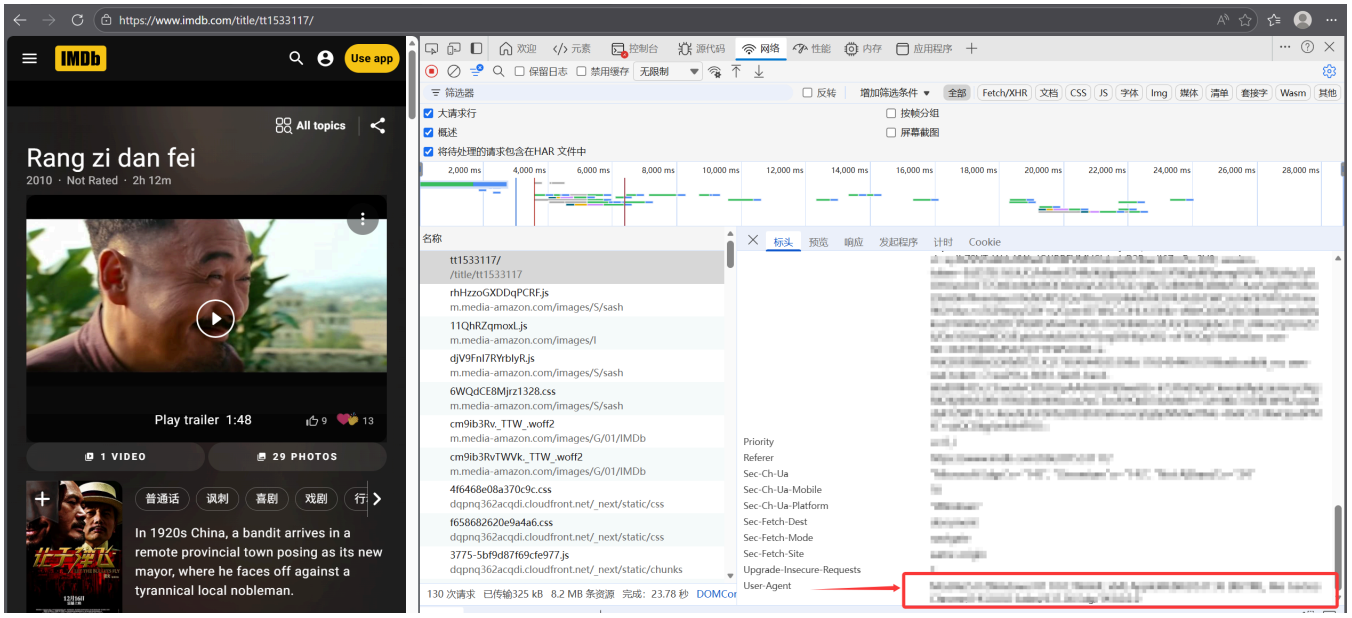
4.Python Code

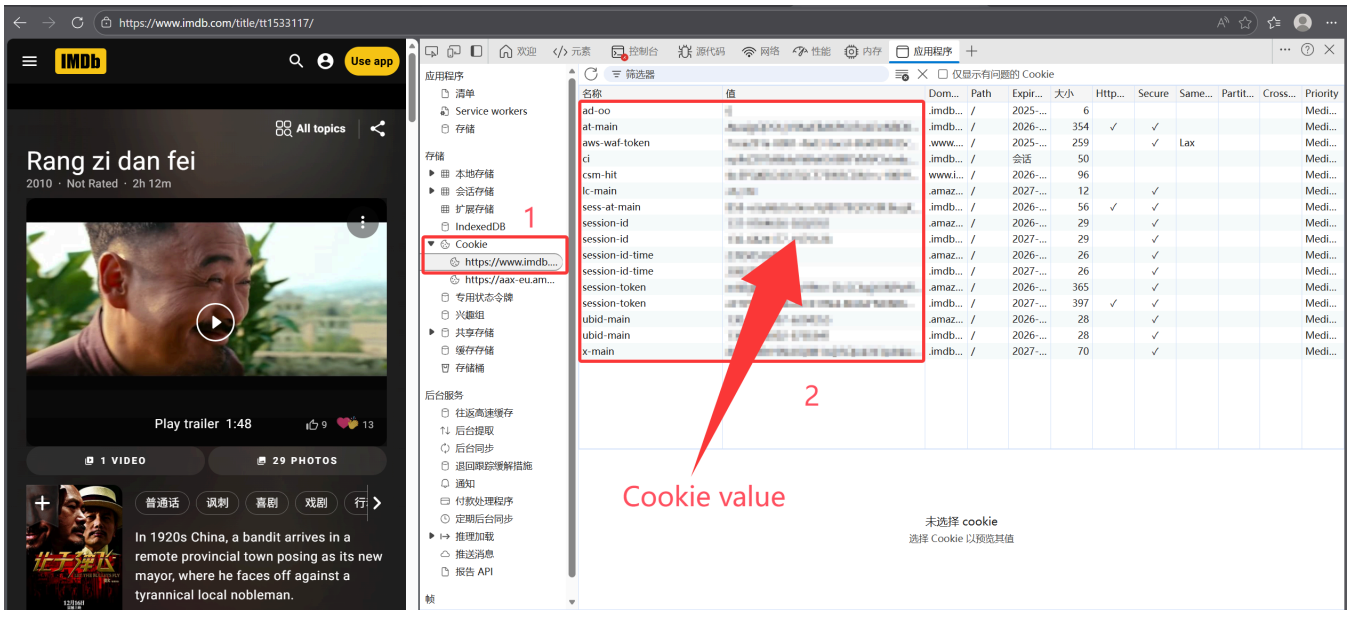
```
#import the necessary packets(use lxml here)
import json
import time
import random
import csv
import requests
from lxml import etree
import pandas as pd
```

```
def log(url):
    with open('database_record_t.txt', 'a') as f:
        f.write(url+'\n')

def get_full_url():

    df = pd.read_csv(
        'movies_tconst.csv',
        encoding='utf-8',
        usecols=['tconst'],  # 只读取 tconst 列（如 tt1234567）
        dtype={'tconst': str}  # 确保 tconst 是字符串类型（避免数字格式丢失前导 t）
    )

    total = len(df)
    print(f"开始处理 {total} 个影片链接...\n")

    for idx, row in df.iterrows():
        tconst = row['tconst']  # 获取当前行的 tconst（如 tt0120338）

        base_url = f"https://www.imdb.com/title/{tconst}/"  # 基础影片页面

        print(f"📌 第 {idx + 1}/{total} 个：{base_url}")

        # 访问影片页面（解析数据）
        get_movie_page(base_url)
        # 降低访问频率（防止影响网站正常运行）
        random_delay = random.uniform(3.5, 4)
        time.sleep(random_delay)
```

```
def get_movie_page(url):
    headers = {}
    cookie = {}
    """
```

```
    这里填写本机的header和该网站的cookie，依然在开发者工具处找（见下面图片）
    """
    with open('database_record_t.txt', 'r') as f:
        datas = f.readline()
    print('检查要下载文件是否已经在库中:')

    print("文件链接为url:", url)
    if url.strip('\n') in datas:
        print('该url已存在,检测下一个url...' + '\n')
    else:
        print('未检测到此条url对应的文件' + '\n' + '程序将继续往下执行')
        pass

        response = requests.get(url, headers=headers, cookies=cookie)
        sec = response.elapsed.seconds
        print(response.status_code, sec)
        html = response.text
        print('得到html')  # 检测程序运行情况
        get_info(html, url)
```





```
def get_info(html, url):
    """
    解析IMDB电影详情页的核心数据
    :param html: 网页HTML源码
    :param url: 目标网页URL（用于错误定位）
    :return: None（直接将数据写入CSV文件）
    """
    # 初始化XPath解析器
    content = etree.HTML(html)

    # 存储从JSON中解析的原始数据
    data = {}

    # ========== 第一步：提取并解析页面中的JSON数据 ==========
    try:
        # 定位__NEXT_DATA__脚本标签（IMDB核心数据存储位置）
        script_json = content.xpath('//script[@id="__NEXT_DATA__" and
@type="application/json"]/text()')[0]
        # 将JSON字符串转为Python字典
        data = json.loads(script_json)

    except IndexError:
        print(f"【错误】未找到__NEXT_DATA__标签，URL: {url}")
    except json.JSONDecodeError as e:
        print(f"【错误】JSON解析失败，URL: {url}，原因：{e}")
    except Exception as e:
        print(f"【错误】处理JSON时发生未知异常，URL: {url}，原因：{e}")

    # ========== 第二步：提取电影基础信息（兼容数据缺失场景） ==========
    # 提取电影唯一标识tconst（IMDB主键）
    try:
        tconst = data.get('props', {}).get('pageProps', {}).get('tconst')
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        tconst = 'no info'

    # 提取核心数据容器（aboveTheFoldData包含电影主要信息）
    try:
        original_data = data.get('props', {}) \
            .get('pageProps', {}) \
            .get('aboveTheFoldData', {})
    except (AttributeError, TypeError):
        original_data = 'no info'

    # 提取原始片名（兼容非字典格式）
    try:
        original_title_text = original_data.get('originalTitleText')
        # 若为字典则取text字段，否则置空
        titles = original_title_text.get('text') if isinstance(original_title_text, dict)
else None
    except (AttributeError, TypeError):
        titles = 'no info'

    # 提取上映年份
```

```
    try:
        original_year = original_data.get('releaseYear', {})
        year = original_year.get('year') if isinstance(original_year, dict) else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        year = 'no info'

    # 提取影片时长（秒转分钟，取整数）
    try:
        runsec = original_data.get('runtime', {})
        runtime = int(runsec.get('seconds')/60) if (isinstance(runsec, dict) and
runsec.get('seconds')) else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        runtime = 'no info'

    # 提取IMDB评分
    try:
        original_rating = original_data.get('ratingsSummary', {})
        rating = original_rating.get('aggregateRating') if isinstance(original_rating, dict)
else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        rating = 'no info'

    # 提取上映国家
    try:
        original_country = original_data.get('releaseDate', {})
        country_text = original_country.get('country', {}) if isinstance(original_country,
dict) else None
        country = country_text.get('text') if isinstance(country_text, dict) else None
    except Exception:  # 替换过窄的IndexError，兼容所有异常
        country = 'no info'

    # ========== 第三步：提取影人关联信息（导演/编剧/演员） ==========
    # 存储影人映射数据（tconst-IMDB_ID-角色类型）
    credit_ids = []
    try:
        # 逐层提取影人列表（每一层兜底为空，避免KeyError）
        principal_credits = data.get('props', {}) \
            .get('pageProps', {}) \
            .get('aboveTheFoldData', {}) \
            .get('principalCreditsV2', [])

        # 遍历影人分组（Director/Writers/Stars）
        for credit_group in principal_credits:
            # 跳过非字典项，防止后续取值报错
            if not isinstance(credit_group, dict):
                continue

            # 提取分组名称（如 Director/Directors/Writers/Stars）
            grouping = credit_group.get('grouping', {})
            grouping_text = grouping.get('text', '').strip()

            # 处理导演分组（D = Director）
            if grouping_text in ['Director', 'Directors']:
```

```
                credits = credit_group.get('credits', [])
                for credit in credits:
                    if not isinstance(credit, dict):
                        continue
                    # 提取导演唯一ID
                    name = credit.get('name', {})
                    person_id = name.get('id')
                    # 过滤空ID，保证数据有效性
                    if person_id and person_id.strip():
                        credit_ids.append({
                            'tconst': tconst,
                            'IMDB_ID': person_id,
                            'credited_as': 'D'  # D=导演
                        })

            # 处理编剧分组（W = Writer）
            elif grouping_text in ['Writer', 'Writers']:
                credits = credit_group.get('credits', [])
                for credit in credits:
                    if not isinstance(credit, dict):
                        continue
                    # 提取编剧唯一ID
                    name = credit.get('name', {})
                    person_id = name.get('id')
                    if person_id and person_id.strip():
                        credit_ids.append({
                            'tconst': tconst,
                            'IMDB_ID': person_id,
                            'credited_as': 'W'  # W=编剧
                        })

            # 处理演员分组（A = Actor）
            elif grouping_text in ['Star', 'Stars']:
                credits = credit_group.get('credits', [])
                for credit in credits:
                    if not isinstance(credit, dict):
                        continue
                    # 提取演员唯一ID
                    name = credit.get('name', {})
                    person_id = name.get('id')
                    if person_id and person_id.strip():
                        credit_ids.append({
                            'tconst': tconst,
                            'IMDB_ID': person_id,
                            'credited_as': 'A'  # A=演员
                        })

    except Exception as e:
        print(f"【错误】提取影人ID失败，URL: {url}，原因：{str(e)}")

    # ========== 第四步：写入CSV文件 ==========
    # 整理电影基础数据
    movie_data = {
```

```
        'tconst': tconst,           # IMDB电影唯一标识
        'titles': titles,           # 原始片名
        'country_fullname': country,  # 上映国家
        'year_released': year,      # 上映年份
        'runtime': runtime,         # 影片时长（分钟）
        'rating': rating            # IMDB评分
    }

    # 写入电影基础数据CSV（追加模式，UTF-8编码避免乱码）
    with open('movie_info_IMDB.csv', 'a', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=movie_data.keys())
        writer.writerow(movie_data)

    # 写入影人映射数据CSV（批量写入，效率更高）
    if credit_ids:  # 仅当有数据时写入，避免空行
        with open('person_mapping_IMDB.csv', 'a', newline='', encoding='utf-8') as f:
            writer = csv.DictWriter(f, fieldnames=['tconst','IMDB_ID','credited_as'])
            writer.writerows(credit_ids)

    # 清空列表，防止后续复用导致数据污染
    credit_ids.clear()

    # 记录爬取日志
    log(url)
    print(f"URL {url} 已处理完成，数据写入CSV\n" + '-' * 60 + '\n\n')
```

```
def main():
    get_full_url()


if __name__ == '__main__':
    main()
```

5.Exercise

This exercise focuses on **checking website crawling rules** (via `robots.txt`) and **locating target data in HTML source code** (using browser developer tools).

Task 1: Find & Analyze `robots.txt` for 2 Websites

Complete the following steps for **Douban Movie** (`https://movie.douban.com/`) and **Southern University of Science and Technology (SUSTech)** (`https://www.sustech.edu.cn/`):

Step 1: Access `robots.txt`

For any website, `robots.txt` is stored in the **root directory** (the main URL without subpaths).

- For Douban Movie: Visit `https://movie.douban.com/robots.txt`
- For SUSTech: Visit `https://www.sustech.edu.cn/robots.txt`

Step 2: Analysis `robots.txt`

Find the **actual core content** and analysis the `robots.txt` for Douban Movie. What is the actual core content of Douban Movie's (https://movie.douban.com/) `robots.txt` file, and specifically which paths are disallowed for crawlers?

Task 2: Locate Movie Data for *Hidden Man* in Douban Page Source Code

Target URL: `https://movie.douban.com/subject/26366496/` (Douban page for *邪不压正 / Hidden Man*)

Step 1: Open the Target Page & Developer Tools

1. Visit the URL in Chrome/Firefox/Edge.
2. Open **Developer Tools**:
    - Shortcut: `F12` (Windows/Linux) or `Cmd + Opt + I` (Mac)
    - Right-click any area on the page → select **Inspect** (检查)

Step 2: Locate Core Movie Data via Search & Inspect

Extract **6 key fields** (title, country, rating, release date, runtime, IMDb primary key) from the HTML code.

```
<script type="application/ld+json">
{
  "@context": "http://schema.org",
  "name": "邪不压正",
  "url": "/subject/26366496/",
  "image": "https://img2.doubanio.com/view/photo/s_ratio_poster/public/p2526297221.webp",
  .......
  "datePublished": "2018-07-13",
  "genre": ["\u5267\u60c5", "\u559c\u5267", "\u52a8\u4f5c"],
  "duration": "PT2H17M",
  "description": "七七事变前夕，华裔青年小亨德勒（彭于晏 饰）从美国远赴重洋，回到阔别十数年之久的北平从医。然而他真正的名字叫李天然，十三岁那年曾亲眼目睹师父一家遭师兄朱潜龙（廖凡 饰）和日本人根本一郎（泽田谦也 饰）...",
  "@type": "Movie",
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingCount": "736977",
    "bestRating": "10",
    "worstRating": "2",
    "ratingValue": "7.0"
  }
}
</script>
```

```
<div id="info">
            <span><span class='pl'>导演</span>: <span class='attrs'><a
href="https://www.douban.com/personage/27227726/" rel="v:directedBy">姜文</a></span></span>
<br/>
            <span><span class='pl'>编剧</span>: <span class='attrs'><a
href="https://www.douban.com/personage/27227726/">姜文</a> / <a
href="https://www.douban.com/personage/27540306/">何冀平</a> / <a
href="https://www.douban.com/personage/27570190/">李非</a> / <a
href="https://www.douban.com/personage/27554152/">孙悦</a> / <a
href="https://www.douban.com/personage/27614555/">张北海</a></span></span><br/>
            <span><span class='pl'>主演</span>: <span class='attrs'><a
href="https://www.douban.com/personage/27219486/" rel="v:starring">彭于晏</a> / <a
href="https://www.douban.com/personage/27213092/" rel="v:starring">廖凡</a> / <a
href="https://www.douban.com/personage/27227726/" rel="v:starring">姜文</a> / <a
href="https://www.douban.com/personage/27243602/" rel="v:starring">周韵</a> / <a
href="https://www.douban.com/personage/27210959/" rel="v:starring">许晴</a> / <a
href="https://www.douban.com/personage/27211222/" rel="v:starring">泽田谦也</a> / <a
href="https://www.douban.com/personage/30271958/" rel="v:starring">安地</a> / <a
href="https://www.douban.com/personage/27481544/" rel="v:starring">史航</a> / <a
href="https://www.douban.com/personage/27547819/" rel="v:starring">李梦</a> / <a
href="https://www.douban.com/personage/27244725/" rel="v:starring">丁嘉丽</a> / <a
href="https://www.douban.com/personage/30388126/" rel="v:starring">陈曦</a></span></span>
<br/>
            <span class="pl">类型:</span> <span property="v:genre">剧情</span> / <span
property="v:genre">喜剧</span> / <span property="v:genre">动作</span><br/>
            <span class="pl">制片国家/地区:</span> 中国大陆<br/>
            <span class="pl">语言:</span> 汉语普通话 / 英语 / 日语 / 法语<br/>
            <span class="pl">上映日期:</span> <span property="v:initialReleaseDate"
content="2018-07-13(中国大陆)">2018-07-13(中国大陆)</span><br/>
            <span class="pl">片长:</span> <span property="v:runtime" content="137">137分钟
</span> / 134分钟(香港)<br/>
            <span class="pl">又名:</span> 侠隐 / Hidden Man<br/>
            <span class="pl">IMDb:</span> tt8434380<br>

</div>
```

Tips

- If you can't find `robots.txt` for a website, it means there are **no explicit crawling restrictions** (but you still need to comply with laws and website terms).
- For Douban's movie page, if the HTML structure is complex, use the **search function** in the `Elements` panel and enter whatever you want to quickly locate the code.